



Introdução à Análise de Algoritmos e Contagem de Primitivas - Aula 2

🕒 Criado em	7 de setembro de 2026 11:24
🏷 Tags	

1. De onde vem a necessidade de analisar um programa?

Na programação, o primeiro objetivo normalmente é fazer um programa **funcionar**. Porém, em Engenharia de Programas, isso não é suficiente.

Um programa pode produzir a resposta correta e, ainda assim, ser inadequado porque:

- demora demais para executar;
- utiliza recursos excessivos;
- exige um hardware muito potente;
- consome muita energia;
- utiliza muita largura de banda;
- não consegue lidar com entradas grandes;
- é difícil de modificar ou manter.

Por isso, um dos princípios básicos da Engenharia apresentados na disciplina é a **eficiência**.

A eficiência procura obter o resultado desejado utilizando uma quantidade adequada de recursos.

No caso de programas, esses recursos podem incluir:

- tempo de processamento;
- capacidade computacional;
- memória;
- energia;
- largura de banda;
- infraestrutura;
- recursos financeiros e humanos durante o desenvolvimento.

Assim, analisar um algoritmo significa, entre outras coisas, tentar descobrir **quanto recurso ele necessita para resolver determinado problema**.

2. Algoritmo, código-fonte e execução

Para entender a análise de complexidade, primeiro precisamos separar alguns conceitos.

Problema

É aquilo que queremos resolver.

Exemplo:

| Dada uma lista de números, encontrar o maior valor.

Algoritmo

É o conjunto de passos utilizados para resolver esse problema.

Código-fonte

É a implementação desse algoritmo em uma linguagem de programação.

Por exemplo, podemos implementar o algoritmo em Python.

Execução

Quando o programa é executado, o computador realiza diversas operações para transformar os dados de entrada em dados de saída.

Podemos representar essa relação assim:

Problema → Algoritmo → Código-fonte → Operações → Resultado

A Engenharia de Programas vai observar principalmente o que acontece nessa última etapa: **quantas operações são necessárias e qual é o custo delas.**

3. Por que não basta medir o tempo diretamente?

Uma maneira aparentemente simples de analisar um programa seria executá-lo e utilizar um cronômetro:

| "O programa levou 2 segundos."

O problema é que esse resultado depende do ambiente em que o programa foi executado.

O mesmo código pode apresentar tempos diferentes dependendo de:

- processador;
- memória;
- sistema operacional;
- versão da linguagem;
- implementação da linguagem;
- outros processos executando simultaneamente;
- características da entrada.

Por exemplo:

Computador A
Programa → 0,8 s

Computador B
Programa → 2,1 s

Isso não significa necessariamente que o código seja diferente.

Por isso, a disciplina procura uma forma mais estruturada de analisar o programa: **contar as operações realizadas pelo código**.

4. O que são primitivas?

Para realizar essa contagem, o professor trabalha com determinadas operações fundamentais chamadas de **primitivas**.

Na abordagem utilizada na disciplina, elas são agrupadas principalmente nas categorias:

- **O — Operações**
- **C — Comparações**
- **A — Atribuições**

O material também apresenta o **F — Função**, formando a ideia de **FOCA**.

A classificação permite transformar um código em uma quantidade mensurável de operações.

Por exemplo:

```
y = a + b
```

Podemos identificar:

```
a + b → 1 operação (O)
y = ... → 1 atribuição (A)
```

Portanto, nessa abordagem simplificada:

```
O = 1
A = 1
C = 0
```

5. Uma observação importante sobre as primitivas

Quando analisamos uma linha como:

```
y = A[i] + B[i]
```

o computador internamente realiza muitas operações de baixo nível envolvendo memória, índices e processamento.

Porém, **não vamos tentar contar todas essas microoperações.**

A disciplina estabelece uma forma padronizada e abstrata de contagem.

Nesse exemplo, podemos considerar:

```
A[i] + B[i]
    ↓
1 operação matemática (0)

y = ...
    ↓
1 atribuição (A)
```

Essa abstração é importante porque permite comparar algoritmos de maneira sistemática.

6. O grande problema dos laços de repetição

Agora chegamos a uma das partes mais importantes da disciplina.

Considere:

```
for i in range(n):
    y = a + b
```

A operação:

```
a + b
```

não acontece apenas uma vez.

Ela acontece **a cada iteração do laço**.

Se:

```
n = 10
```

o bloco será executado 10 vezes.

Se:

```
n = 1.000
```

será executado 1.000 vezes.

Se:

```
n = 1.000.000
```

será executado 1.000.000 de vezes.

Portanto, o número de operações realizadas pelo programa passa a depender do tamanho da entrada.

É justamente aqui que começa a surgir a ideia de **complexidade de tempo**.

7. Entendendo o **for**

O professor começa analisando o caso:

```
for i in range(n):  
    ...
```

Podemos interpretar:

```
x = 0  
y = n  
k = 1
```

Assim, o índice assume os valores:

```
i = 0, 1, 2, 3, ..., n - 1
```

Logo, existem **n iterações do bloco**.

Mas existe um detalhe importante:

A execução de um `for` não consiste apenas em executar o bloco interno.

Também existe o trabalho necessário para **gerenciar o próprio laço**.

8. Gestão do laço

O professor analisa o funcionamento do laço em etapas.

Para:

```
for i in range(n):  
    ...
```

podemos pensar, de forma simplificada:

Inicialização

Primeiro são definidos os parâmetros do laço:

```
valor inicial  
valor final  
incremento
```

Depois o contador é inicializado.

No material, isso é representado pelo grupo:

```
g1 = (0, 0, 0, 4)
```

Esse vetor representa as quantidades de primitivas utilizadas na inicialização.

O valor `4` corresponde às quatro atribuições consideradas nessa etapa.

9. A cada iteração

Depois da inicialização, o laço precisa verificar se ainda deve continuar.

A estrutura é aproximadamente:

```
Inicializa
  ↓
Verifica condição
  ↓
Ainda pode continuar?
├─ Sim → incrementa → verifica novamente
└─ Não → encerra
```

Na análise do professor, a cada repetição são consideradas principalmente:

- uma comparação;
- uma operação;
- uma atribuição.

Isso é representado por:

```
gr = (0, 1, 1, 1)
```

Ou seja:

```
F = 0
O = 1
C = 1
A = 1
```

para cada repetição considerada na gestão do laço.

10. Por que aparece **n + 1** ?

Essa é uma parte que merece bastante atenção.

Se temos:

```
for i in range(n):
```

o corpo do laço executa n vezes.

Porém, a condição precisa ser verificada **uma última vez para descobrir que o laço terminou**.

Imagine:

```
i = 0 → continua  
i = 1 → continua  
i = 2 → continua  
...  
i = n-1 → continua  
i = n   → para
```

A condição foi verificada:

$n + 1$ vezes

Por isso aparecem expressões como:

$n + 1$

na contagem das primitivas.

Essa diferença entre **n iterações do corpo** e **$n + 1$ verificações da condição** é uma das coisas que você precisa prestar bastante atenção.

11. FOCA do laço

O professor representa o custo de gestão do laço como:

$FOCA_loop = g_1 + Xg_r$

Onde:

- g_1 representa o custo de inicialização;
- g_r representa o custo de cada repetição;
- X representa quantas vezes essa parte da gestão é executada.

Para o caso:

```
for i in range(n):
```

temos:

```
X = n + 1
```

Então:

```
FOCA_loop = (0,0,0,4) + (n+1)(0,1,1,1)
```

Realizando as operações:

```
FOCA_loop = (0, n+1, n+1, n+5)
```

Portanto:

```
O = n + 1  
C = n + 1  
A = n + 5
```

Esse resultado representa **a gestão do laço**, não necessariamente todas as operações existentes dentro do corpo.

12. Abordagem simplificada

O professor também apresenta uma maneira mais simples de analisar o laço.

Em vez de considerar que cada tipo de primitiva possui um tempo diferente, podemos simplesmente somar todas as primitivas.

Assim:

```
P_loop(n) = O(n) + C(n) + A(n)
```

Substituindo:

$$P_loop(n) = (n+1) + (n+1) + (n+5)$$

Logo:

$$P_loop(n) = 3n + 7$$

Isso significa que a gestão desse laço realiza:

$3n + 7$ primitivas

na abordagem simplificada.

13. Abordagem detalhada

Porém, as primitivas não necessariamente levam exatamente o mesmo tempo.

Uma operação matemática pode ter um custo diferente de uma atribuição, que por sua vez pode ter um custo diferente de uma comparação.

Por isso, o professor define tempos médios:

- τ_o → tempo médio de uma operação;
- τ_c → tempo médio de uma comparação;
- τ_a → tempo médio de uma atribuição.

Então podemos estimar:

$$T_loop(n) = O(n)\tau_o + C(n)\tau_c + A(n)\tau_a$$

Substituindo as quantidades encontradas:

$$T_loop(n) = (n+1)\tau_o + (n+1)\tau_c + (n+5)\tau_a$$

Essa é uma estimativa mais detalhada do tempo gasto na **gestão do laço**.

14. E de onde vêm τ_o , τ_c e τ_a ?

Esses valores não são simplesmente inventados.

Precisamos obtê-los por meio de **calibração experimental**.

A ideia é executar experimentos para estimar quanto tempo, em média, determinada operação leva no ambiente utilizado.

Esse ambiente inclui:

```
Hardware
+
Sistema operacional
+
Versão do Python
+
Ambiente de execução
```

Portanto, a disciplina combina duas ideias:

```
CONTAGEM
↓
Quantas operações existem?

CALIBRAÇÃO
↓
Quanto cada tipo de operação custa?

RESULTADO
↓
Estimativa do tempo
```

15. A função de tempo

De maneira geral, podemos representar o tempo de execução de um algoritmo por uma função:

$f(n)$

O n representa o tamanho da entrada ou algum parâmetro que determina o tamanho do problema.

A função pode ser formada pela soma dos custos das diferentes primitivas:

$$f(n) = F_{\tau}f + O_{\tau}o + C_{\tau}c + A_{\tau}a$$

A forma exata depende do código que estamos analisando.

O ponto fundamental é:

O tempo de execução passa a ser estudado como uma função do tamanho do problema.

16. Por que o formato $a + bn$ é importante?

Depois de contar as operações e organizar a expressão, podemos chegar a algo como:

$$f(n) = a + bn$$

Aqui:

- a representa custos que não dependem de n ;
- b representa o custo que cresce proporcionalmente a n .

Por exemplo:

$$f(n) = 20 + 5n$$

Para:

$$n = 10$$

temos:

$$f(10) = 70$$

Para:

$$n = 1.000$$

temos:

$$f(1000) = 5.020$$

Percebemos que, conforme n cresce, o termo relacionado a n passa a dominar o comportamento da função.

17. Complexidade de tempo

É nesse ponto que chegamos à ideia de **complexidade de tempo**.

Não estamos interessados somente em saber:

| "Quanto tempo esse programa levou nesta execução?"

Queremos compreender:

| **"Como o custo de execução cresce quando o tamanho do problema aumenta?"**

Essa é uma diferença fundamental.

Por exemplo:

Programa A:
 $f(n) = 10 + 5n$

e

Programa B:
 $f(n) = 3 + 2n^2$

Para entradas pequenas, talvez a diferença não seja tão importante.

Mas, conforme n aumenta, o comportamento dos dois programas começa a ficar muito diferente.

Por isso, analisar complexidade significa observar principalmente **a forma de crescimento do custo**.

18. A ideia da notação Big-O

A notação **Big-O** é utilizada para representar o comportamento assintótico do crescimento de uma função.

Se temos:

$$f(n) = 10 + 5n$$

o termo que cresce com n é:

n

Então dizemos que o crescimento é:

$$O(n)$$

Isso é chamado de **complexidade linear**.

É importante entender que:

| $O(n)$ não significa simplesmente "o programa demora n segundos".

Significa que estamos classificando **como o custo cresce em função de n** .

19. Por que as constantes são desconsideradas?

Considere:

$$f(n) = 100 + 20n$$

e:

$$g(n) = 5 + 2n$$

Ambas possuem crescimento linear.

Os valores 100, 20, 5 e 2 influenciam o tempo real, mas não mudam a classificação do crescimento.

Por isso, na análise assintótica, observamos a ordem de crescimento:

$$\begin{aligned} 100 + 20n &\rightarrow O(n) \\ 5 + 2n &\rightarrow O(n) \end{aligned}$$

O objetivo da Big-O é justamente abstrair detalhes constantes e destacar **como o custo cresce quando a entrada aumenta**.

20. Um cuidado importante: contagem de primitivas $\neq$ Big-O

Esses conceitos estão relacionados, mas não são a mesma coisa.

Contagem de primitivas

Pergunta:

| Quantas operações o código realiza?

Podemos obter:

$$3n + 7$$

Função de tempo

Considera o custo das diferentes operações:

$$\begin{aligned} T(n) = & \\ & (n+1)\tau_o + \\ & (n+1)\tau_c + \\ & (n+5)\tau_a \end{aligned}$$

Complexidade

Observa o comportamento de crescimento:

$O(n)$

Portanto:

```
Contagem
  ↓
Função de custo/tempo
  ↓
Comportamento assintótico
  ↓
Complexidade
```

21. Exemplo completo

Considere:

```
for i in range(n):
    y = a + b
```

Temos duas coisas diferentes para analisar.

Gestão do laço

O professor encontrou:

```
O = n + 1
C = n + 1
A = n + 5
```

Corpo do laço

A instrução:

```
y = a + b
```

possui, na contagem simplificada:

```
O = 1  
A = 1
```

Como o corpo é executado n vezes:

```
O = n  
A = n
```

Portanto, quando quisermos analisar **o programa inteiro**, precisamos somar o custo da gestão do laço com o custo das instruções executadas dentro dele.

Essa distinção é muito importante.

Não devemos confundir:

custo de administrar o laço

com

custo do algoritmo executado dentro do laço.

22. O que isso tem a ver com o mercado?

Essa análise não existe apenas para fazer exercícios acadêmicos.

Imagine uma empresa que possui um sistema que processa imagens.

Uma versão do algoritmo consegue processar:

```
100 imagens → rapidamente
```

Mas quando recebe:

```
100.000 imagens
```

o sistema fica extremamente lento.

A equipe pode então investigar:

- quantas operações o algoritmo realiza?
- quais operações são repetidas?

- existem laços desnecessários?
- o algoritmo cresce linearmente?
- existe um laço dentro de outro?
- determinada operação poderia ser reduzida?
- a implementação utiliza recursos demais?

Esse tipo de análise ajuda uma equipe a tomar decisões técnicas antes de simplesmente "comprar um computador mais potente".

23. A conexão com os projetos da disciplina

Isso também explica por que os projetos da disciplina utilizam Python e processamento de imagens.

Vocês não estarão apenas desenvolvendo um programa.

O objetivo será observar o comportamento dele de diferentes maneiras.

A sequência apresentada pelo professor é:

```
1. Desenvolver o programa
  ↓
2. Executar e medir experimentalmente
  ↓
3. Analisar matematicamente
  ↓
4. Comparar os resultados
  ↓
5. Estudar o comportamento da complexidade
```

Assim, vocês terão a oportunidade de comparar:

o que o computador realmente mediu

com

o que a análise matemática previu.

Essa comparação é uma das partes mais importantes da proposta da disciplina.

24. O que você precisa realmente entender desta aula

Se você compreender estes pontos, terá uma boa base para avançar:

1. Programa não precisa apenas funcionar

Ele também deve ser analisado quanto à eficiência e ao uso de recursos.

2. Algoritmo e código não são exatamente a mesma coisa

Um algoritmo pode possuir diferentes implementações.

3. O código realiza operações

Na disciplina, essas operações são abstraídas em primitivas como:

```
F – Função  
O – Operação  
C – Comparação  
A – Atribuição
```

4. Laços repetem operações

Por isso, o custo de um programa pode crescer conforme `n` aumenta.

5. É necessário contar as repetições

No caso de:

```
for i in range(n):
```

o corpo executa `n` vezes, enquanto determinadas verificações da gestão do laço aparecem `n + 1` vezes.

6. Podemos contar primitivas

Por exemplo:

```
P_loop(n) = 3n + 7
```

7. Podemos estimar o tempo

Considerando o custo médio de cada tipo de primitiva:

$$T(n) = \\ 0\tau_o + \\ C\tau_c + \\ A\tau_a$$

8. O tempo depende do ambiente

Os valores de τ_o , τ_c e τ_a precisam ser estimados experimentalmente para o ambiente considerado.

9. Complexidade observa o crescimento

Não estamos apenas perguntando:

| "Quanto demorou?"

Estamos perguntando:

| "Como o custo cresce quando o tamanho do problema aumenta?"

10. Big-O representa esse crescimento

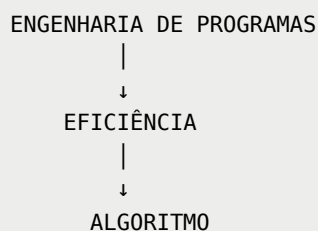
Uma função como:

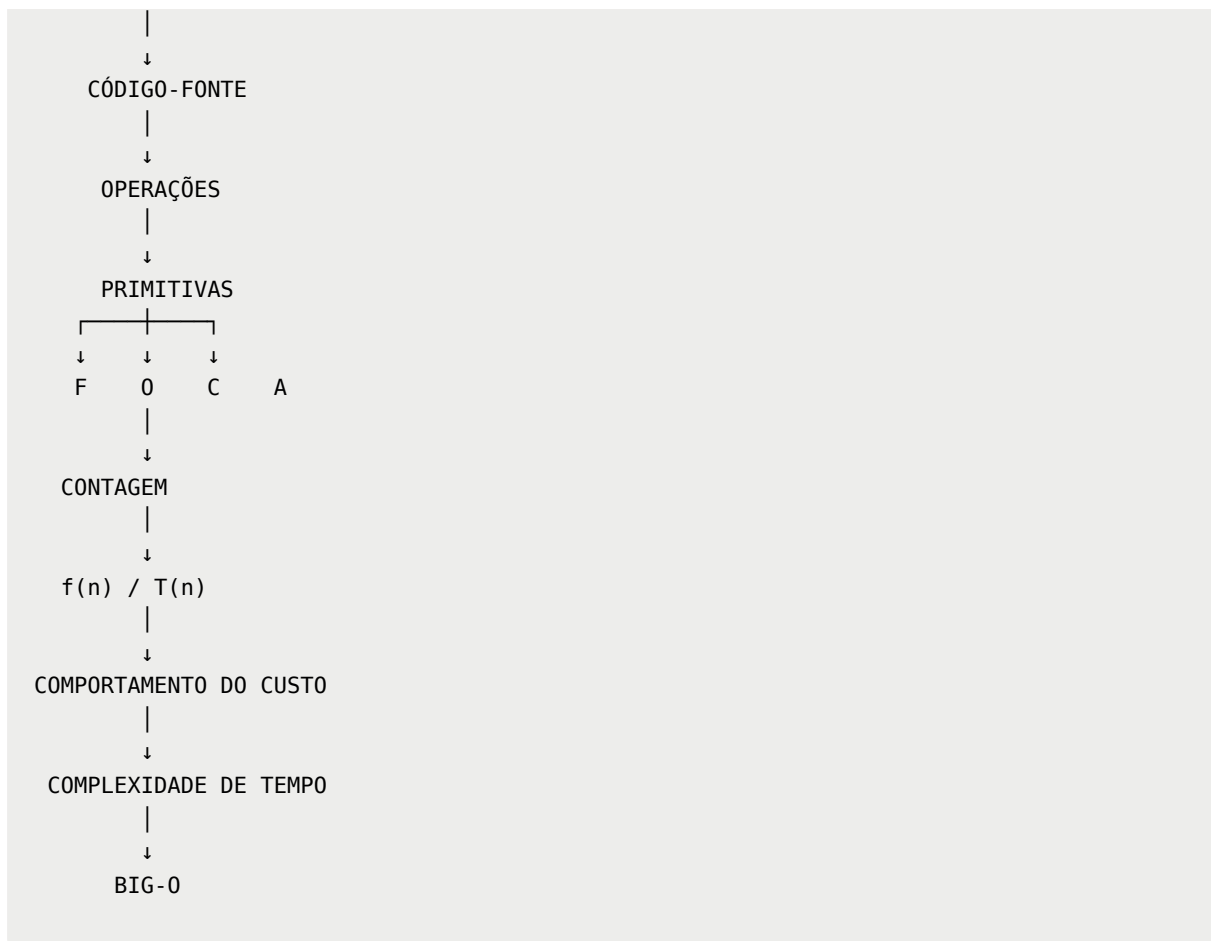
$$f(n) = a + bn$$

possui crescimento linear:

$$O(n)$$

25. Mapa mental da aula





A ideia central pode ser resumida em uma frase:

A análise de algoritmos procura entender como o custo de execução de um programa cresce conforme o tamanho do problema aumenta, utilizando uma contagem padronizada de operações e, quando necessário, uma estimativa do tempo associado a cada tipo de operação.